Enunciado de PCD - 2026/27
IscoT:

Versão 4 *- 2 de novembro de 2026

Resumo

O projeto pretende modelar um sistema de gestão doméstica através de domótica. Existirão sensores, decisores e atuadores que se ligarão a um servidor central, que fará a gestão de toda a comunicação no sistema distribuído. O componente central será um *message broker*[1], aqui designado *IscoT*. O intuito é, com o apelo da atividade prática, desenvolver melhor a aprendizagem do conteúdo ministrado na UC.

Message brokers são amplamente usados na indústria informática em cenários de processamento paralelo e assíncrono, inclusive em cenários de automação doméstica, como o que simularemos neste projeto.

Para bem servir os propósitos de aprendizagem, as características de utilização da aplicação foram adaptadas, de maneira que aprimore o cenário de aplicação de questões habituais nos domínios da concorrência e distribuição.

1 Descrição genérica do projeto

O projeto consiste no desenvolvimento de um sistema de encaminhamento de eventos/mensagens, através de um *message broker*, com origem em clientes remotos, que podem ser de vários tipos, sensores, decisores ou atuadores, e que poderão emitir eventos a qualquer momento. Exemplos de protocolos populares neste âmbito são MQTT (este, mais comumente usado para domótica), Kafka, RabbitMQ, entre muitos outros, mas no presente projeto vamos usar um protocolo próprio.

*__Nota:__ o enunciado pode ser alterado para melhorar a clareza ou adequar-se melhor às necessidades de aprendizagem. Qualquer nova versão será publicada no moodle e notificada por email.

[1] `https://en.wikipedia.org/wiki/Message_broker`

O funcionamento se dá, em linhas gerais, da seguinte forma: um servidor que centraliza a gestão de filas de eventos (que podem ser entendidos como mensagens) preenchidas de informação a ser processada. Por outro lado, uma série de clientes remotos que se ligam ao servidor consomem eventos para processar ou publicam eventos com dados para serem processados por outros clientes. Os eventos são separados no servidor por tópicos, que são agrupamentos de eventos de um mesmo tema a ser tratado. Um cliente pode publicar eventos exclusivamente de um tópico, e/ou consome eventos de um outro tópico. Em particular, os clientes sensores apenas publicam eventos do tipo *Sensor*, os decisores consomem do tipo *Sensor* e publicam do tipo *Highlevel*, e os atuadores apenas consomem eventos *Highlevel*.

Elencam-se de seguida as tarefas mais importantes no funcionamento de um sistema deste tipo.

- Configuração do servidor: o servidor recebe ligações de sensores de diversos tipos que correspondem a leituras do ambiente da casa onde estão instalados. Além destes, o servidor também aceita ligações de decisores, interessados nos dados produzidos pelos sensores. Os decisores emitirão ordens de ações de alto nível. Serão os atuadores que receberão e interpretarão estas ações de alto nível, que originarão acionamento e mudanças no estado de componentes físicos como lâmpadas, ar condicionado ou sistemas de rega.

- Receção das leituras dos sensores: o servidor recebe eventos que contêm sinais emitidos pelos sensores (classes `GroundMoistureSensorEvent`, `HumiditySensorEvent`, `LightSensorEvent` e `TemperatureSensorEvent`, todas subclasses de `SensorEvent`) em intervalos de tempo arbitrários (entre 1 segundo e várias horas). Os sinais devem ser tratados como eventos a serem disponibilizadas para acesso pelos decisores nelas interessados.

- Consumo dos eventos emitidos: Os decisores tomarão em conta leituras dos sensores e emitirão ordens de alto nível (classes `AreaIrrigationOrderEvent`, `AreaLightingOrderEvent`, `AreaTemperatureOrderEvent` e `AreaVentilationOrderEvent`, todas subclasse de `HighLevelOrderEvent`), que serão processadas pelos atuadores. Pela sua parte, os atuadores devem subscrever um tópico e uma área adequados para receber os eventos relacionados com o(s) dispositivo(s) que estão a controlar.

- Ciclo de vida da gestão de eventos:

  Publicação: o evento é publicado numa fila para ser consumido pelos

clientes interessados no seu tópico correspondente (decisores ou atuadores), e só é consumido quando chega à cabeça desta fila.

Remoção: O evento só deve ser removido da fila quando todos os clientes subscritos no tópico correspondente confirmarem o seu consumo, ou após um tempo de espera po pré-definido. Ao consumir um evento, um cliente atuador ou decisor decide como utilizar a informação para alterar o comportamento de um dispositivo da *smarthome* simulada.

Próximo evento: O servidor avança para o próximo evento disponível em cada tópico quando o evento à cabeça da fila é considerado consumido. Em caso de esgotamento da fila, os clientes atuadores esperam até que haja novo evento disponível.

1.1 Responsabilidades do Servidor

O servidor recebe as leituras de estado dos sensores, bem como ordens de alto nível (dos decisores), e publica-as como eventos nos tópicos correspondentes, para serem consumidos pelos clientes decisores e atuadores, além de gerir as filas associadas a cada tópico de acordo com as condições estabelecidas para seu funcionamento. Cada cliente deve ter uma thread dedicada no servidor, que gere as interações com ele, incluindo (mas não limitado a) envio e receção de eventos.

1.2 Responsabilidades do Cliente

No arranque do cliente, este liga-se automaticamente ao servidor, indicando o seu tipo e a área da casa onde opera (valores possíveis: sala, querto, cozinha, wc e jardim). Estes parâmetros, bem como o endereço e porto do servidor, devem ser recebidos como argumentos de execução do *main*[2].

As automações desenvolvidas com o consumo de eventos deverão ser demonstradas via GUI, como nos seguintes exemplos:

- se o sensor de luz indica baixa luminosidade numa divisão, o cliente decisor define que se devem ligar as luzes, e deve esperar que todos os atuadores encarregues da iluminação acendam a lâmpada que controlam;

- se o sensor de humidade no ar avisar que a o ar está húmido, o agente decisor da ventilação define uma ordem, e os atuadores ligam ventila-

[2]https://docs.oracle.com/javase/tutorial/essential/environment/cmdLineArgs.html

dores ou ar condicionado no modo ventilação, apenas se não estiver já em algum dos modos aquecimento ou arrefecimento;

- se o sensor de humidade no solo avisar que a terra está seca, o agente decisor da rega define uma ordem de irrigação, e os atuadores ligam a irrigação do jardim;
- se o termómetro deteta a ultrapassagem de dada temperatura, espoleta a ligação de um ar condicionado, para aquecer ou arrefecer;
- outros comportamentos que sejam pertinentes aos sensores existentes.

1.3 Histórico de ações

A cada ação tomada por um cliente atuador, este adiciona a ação tomada a um histórico verificável pelo utilizador de todas as ações tomadas pelo atuador desde a última vez que foi ligado. Por exemplo: uma lâmpada que tenha sido desligada, deverá conter na listagem de ações tomadas uma mensagem como "Lâmpada desligada - HH:MM:SS", onde HH:MM:SS é o timestamp do horário da ação.

2 Requisitos do projeto

O projeto é uma ferramenta de aprendizagem, pelo que deve servir para adquirir conhecimentos em vários tópicos do programa de PCD, que passamos a elencar e que devem ser escrupulosamente seguidos.

Todos os mecanismos de coordenação devem ser desenvolvidos pelo próprio grupo, não devendo ser usados os equivalentes disponibilizados nas bibliotecas padrão do Java.

2.1 Identificação do dispositivo e seu tipo

Ao iniciar a aplicação, o cliente inscreve-se no servidor, que criará para ele um identificador de dispositivo, que deve ser único em todo o sistema. A área da casa, bem como o tipo de cliente devem ser enviados ao servidor aquando da ligação inicial. Se o tipo não existir a ligação é recusada. O endereço e o porto do servidor, bem como os identificadores do dispositivo e de seu tipo são dados como argumentos ao programa do cliente atuador.
java iscot { IP PORT AreaDispositivo TipoDispositivo }

2.2 Ciclo de vida de um evento

Após o servidor disponibilizar o evento como consumível para os clientes atuadores interessados, o evento é então consumido por estes. O consumo do evento por cada cliente deve ser registado pelo servidor, com o auxílio de um `CountDownLatch`. Só após **todos** os clientes terem consultado o evento é que o servidor disponibilizará o próximo evento do tópico.

Para o controlo de quais dos clientes já responderam corretamente ao consumo do evento publicado pelo tópico, deve-se utilizar um `CountDownLatch`, cuja API será:

```
1 public class ModifiedCountdownLatch {
2         public ModifiedCountdownLatch(int count);
3         public synchronized void countdown() ;
4         public synchronized void await(int waitPeriod)
              ↪ throws InterruptedException ;
5 }
```

2.3 Extra: Receção de confirmações de execução

Após o consumo de cada um dos eventos e seu processamento pelos clientes destinatários, pode ser relevante, em alguns casos, o cliente atuador enviar ao servidor um evento que confirme o correto processamento da ordem. O servidor deverá aguardar pela receção de todos os eventos de confirmação relacionadas a cada tópico antes de registar o pleno sucesso da interação com os clientes atuadores. As respostas devem seguir o seguinte formato, e serem transmitidas como instância da classe `ReceptionConfirmationMessage` :

```
{
    event: "evento-recebido"
    deviceId: "id-do-cliente-atuador",
    status: "OK"/"NOK"
}
```

3 Fases de desenvolvimento

Nesta secção sugere-se um encadeamento de fases que permitem um desenvolvimento sustentado deste projeto.

1. **GUI**: ambientar-se aos vários componentes gráficos disponibilizados para o desenvolvimento do projeto: componentes que simulam os sensores (luz, temperatura, chuva, etc) e os atuadores (luzes, ar condicionado, rega, etc). .

Desenvolvimento da estrutura de tópicos e áreas: desenvolvimento de uma estrutura adequada para manter os eventos correspondentes aos tópicos de eventos: neste caso apenas com os valores `SENSOR`, `DECIDER` e `ACTUATOR`. Cada evento deve também prever a identificação da área, que no caso atual apenas deve prever os valores `WC`, `KITCHEN`, `BEDROOM`, `LIVINGROOM` e `GARDEN`.

2. **Criação da hierarquia de eventos**: criar uma estrutura inicial das eventos previstos no sistema, de acordo com a estrutura de classes descrita acima. Sugere-se a criação de uma classe de topo `Event`, abstrata, que seja superclasse de todas as outras, vem como a criação de `RegistrationEvent` para o registo inicial dos clientes.

3. **Simular a representação das eventos nas diferentes janelas da GUI**: exibir eventos exemplo na várias janelas existentes, no caso dos atuadores, e gerar eventos, a exibir na consola, a partir das janelas dos sensores.

4. **Criação do servidor e ligação inicial dos clientes**: desenvolver uma funcionalidade inicial de ligação no modelo cliente-servidor.

5. **Troca de eventos entre clientes e servidor**: desenvolvimento das classes necessárias ao envio de eventos, através de canais de objetos, e sua aplicação na comunicação entre o servidor e clientes.

6. **Troca simples de eventos**: desenvolver uma versão básica do sistema de encaminhamento de eventos em que existe apenas um atuador, um decisor e um sensor, para teste inicial das funcionalidades de manutenção e encaminhamento de eventos.

7. **Processamento das consultas dos eventos:** aplicação do *CountDownLatch* desenvolvido na gestão das consultas dos eventos em cada tópico.

8. **Extra: Desenvolvimento do consumo confirmado dos eventos de alto nível**: desenvolvimento de funcionalidade de confirmação da execução de ações por parte dos clientes atuadores.

4 Entregas

Para favorecer o início atempado do desenvolvimento do projeto, existe uma entrega intermédia. Pede-se que seja feita uma apresentação das fases

iniciais (fases 1 a 3), referentes a funcionalidades ainda sem concorrência ou distribuição, no turno de laboratório da semana de 10 a 16 de novembro, onde deverá ser feita uma breve apresentação do trabalho feito. Esta entrega é obrigatória e eliminatória, e feita presencialmente.

A entrega final será às 09:00 de 14.12.2025, em página no Moodle a disponibilizar oportunamente. As discussões finais, de que alguns alunos poderão eventualmente ser dispensados, decorrerão nos laboratórios de 15 a 18 de dezembro e, se necessário, estender-se-ão eventualmente para os dias seguintes.

O modelo da avaliação é o habitual: o projeto é obrigatório mas não conta diretamente para a nota, podendo apenas limitar a nota final, conforme critérios apresentados na primeira aula e constante da FUC.

5 Avaliação

Todas as fases comuns descritas na secção 3 devem ser implementadas, pois o projeto é primordialmente uma ferramenta de aprendizagem e essas fases correspondem a partes importantes da matéria a aprender e a ser avaliada.

As tarefas opcionais podem levar a uma avaliação maximizada do projeto.

Caso haja lacunas menores nesta implementação, os alunos poderão mesmo assim ter aprovação com uma valoração qualitativa mais baixa.

6 Histórico de versões

1 Versão inicial

2 Melhor definição das tarefas pretendidas. Para facilitar a compreensão, passou a usar-se o termo 'Evento' ao longo do texto, substituindo a anterior utilização de 'mensagem', para evitar ambiguidades. Definição das fases a apresentar na apresentação intermédia.

3 Correção dos nomes das classes dos eventos, para serem congruentes com o termo evento.

4 Supressão da fase de desenvolvimento 2, que na prática já estava resolvida no código base partilhado, através dos enumerados `Areas` e `Type`.